

Exercício 1: Gerenciamento de Recursos com yield e Teardown

Contexto: Você está desenvolvendo um sistema que precisa manipular arquivos de log temporários durante os testes. É fundamental que cada teste tenha seu próprio caminho de arquivo e que, ao final de cada execução, o arquivo seja removido para não poluir o ambiente.

Tarefa:

1. Crie uma fixture chamada `log_file` que:
 - Defina o caminho para um arquivo chamado `test_log.txt`.
 - Forneça o caminho do arquivo (string ou `Path`) para o teste usando `yield`.
 - Após o teste (Teardown), verifique se o arquivo existe e o remova do sistema de arquivos usando `os.remove`.
2. Escreva um teste `test_escrita_log` que utilize essa fixture para criar e escrever a frase "Teste de log" no arquivo. O teste deve verificar se o arquivo foi realmente criado no disco.

CrITÉrios de Avaliação:

- Uso correto do `yield` para separar Setup e Teardown.
 - Garantia de que o arquivo criado pelo teste seja removido após a execução.
-

Exercício 2: Escopos, autouse e Organização com conftest.py

Contexto: Em um projeto de grande escala, conexões pesadas com banco de dados não devem ser reiniciadas a cada teste, mas sim uma vez por sessão. Além disso, você precisa garantir que um "limpador de cache" rode antes de cada função de teste sem precisar chamá-lo manualmente.

Tarefa:

1. Estruture a seguinte hierarquia de diretórios:

```
projeto/  
├── conftest.py  
└── test_db.py
```

2. No `conftest.py`:
 - Crie a fixture `db_connection` com `scope="session"`. Ela deve retornar a string "Conexão Global Estabelecida".
 - Crie a fixture `clean_cache` com `autouse=True`. Ela deve imprimir no console "Cache limpo antes do teste".

3. No `test_db.py`:

- Escreva dois testes simples que recebam `db_connection` e verifiquem o seu valor.
- Execute os testes no terminal com `pytest -s` e observe a ordem de execução dos logs.

Critérios de Avaliação:

- Aplicação correta do escopo `session`.
 - Uso do `autouse=True` para tarefas transversais.
 - Organização adequada do arquivo `conftest.py`.
-

Exercício 3: Parametrização Dinâmica de Fixtures

Contexto: É necessário validar o controle de acesso de uma aplicação para diferentes papéis de usuários. Em vez de duplicar funções de teste, você deve parametrizar a própria fixture.

Tarefa:

1. Crie uma fixture chamada `user_data` que utilize o argumento `params` para alternar entre três dicionários:
 - `{"role": "admin", "access": True}`
 - `{"role": "editor", "access": True}`
 - `{"role": "guest", "access": False}`
2. Injete `request` na fixture e acesse os dados via `request.param`.
3. Escreva o teste `test_access_control` que receba `user_data` e verifique:
 - Se `role` for `"guest"`, a chave `access` deve ser `False`.
 - Caso contrário, deve ser `True`.

Critérios de Avaliação:

- Uso correto do argumento `params` na fixture.
 - Uso da palavra reservada `request` e do atributo `request.param`.
-

Exercício 4: Desafio de Refatoração e Isolamento

Contexto: Você herdou o seguinte código de teste mal estruturado, que cria dependências manuais e não garante a limpeza de ambiente caso ocorra uma falha:

```
# Código legado para refatorar:  
import os
```

```
def test_processamento_de_dados():  
    # Setup manual  
    os.environ["APP_ENV"] = "testing"  
    banco_falso = ["item1", "item2"]  
  
    # Execução  
    banco_falso.append("item3")  
  
    # Assertões  
    assert os.environ["APP_ENV"] == "testing"  
    assert len(banco_falso) == 3  
  
    # Teardown manual (se a assertão falhar, este código nunca  
    executa!)  
    del os.environ["APP_ENV"]
```